

J Karthik
192472136
CSA6502
GEN - AI

Module 1

Experiment 1: Install Python, Anaconda/Jupyter Notebook, and AI Libraries

Install Python, Anaconda/Jupyter Notebook, and the required AI libraries (NumPy, Pandas, Matplotlib, and Scikit-learn). Verify the installation by importing each library and displaying its version.

Aim

To install Python, Jupyter Notebook, and the required AI libraries (NumPy, Pandas, Matplotlib, and Scikit-learn) and verify their installation by displaying their versions.

Algorithm

- Import the required libraries.
- Display the version of each library using the `__version__` attribute.
- Verify that all libraries are successfully installed.

Program

```
import numpy as np
import pandas as pd
import matplotlib
import sklearn

print("NumPy Version:", np.__version__)
print("Pandas Version:", pd.__version__)
print("Matplotlib Version:", matplotlib.__version__)
print("Scikit-learn Version:", sklearn.__version__)
```

Input

No user input is required.

Output

NumPy Version: 2.0.2

Pandas Version: 2.2.2

Matplotlib Version: 3.10.0

Scikit-learn Version: 1.6.1

Result

The required AI libraries were successfully imported, and their versions were displayed, confirming successful installation.

Experiment 2: Matrix Operations using NumPy

A company stores sales data in matrix form. Write a Python program using NumPy to perform matrix addition, subtraction, multiplication, transpose, and inverse of a matrix.

Aim

To perform matrix addition, subtraction, multiplication, transpose, and inverse using NumPy.

Algorithm

- Import the NumPy library.
- Create two matrices.
- Perform matrix addition.
- Perform matrix subtraction.
- Perform matrix multiplication.
- Find the transpose of a matrix.
- Find the inverse of a matrix.
- Display all the results.

Program

```
import numpy as np

A = np.array([[2, 4],
              [6, 8]])

B = np.array([[1, 3],
              [5, 7]])

print("Matrix A:")
print(A)

print("\nMatrix B:")
print(B)

print("\nAddition:")
print(A + B)

print("\nSubtraction:")
print(A - B)

print("\nMultiplication:")
print(np.dot(A, B))

print("\nTranspose of A:")
print(A.T)

print("\nInverse of A:")
print(np.linalg.inv(A))
```

Input

Matrix A = $\begin{bmatrix} 2 & 4 \\ 6 & 8 \end{bmatrix}$

Matrix B = $\begin{bmatrix} 1 & 3 \\ 5 & 7 \end{bmatrix}$

Output

Addition:

$\begin{bmatrix} 3 & 7 \end{bmatrix}$

$\begin{bmatrix} 11 & 15 \end{bmatrix}$

Subtraction:

$\begin{bmatrix} 1 & 1 \end{bmatrix}$

$\begin{bmatrix} 1 & 1 \end{bmatrix}$

Multiplication:

$\begin{bmatrix} 22 & 34 \end{bmatrix}$

$\begin{bmatrix} 46 & 74 \end{bmatrix}$

Transpose:

$\begin{bmatrix} 2 & 6 \end{bmatrix}$

$\begin{bmatrix} 4 & 8 \end{bmatrix}$

Inverse:

$\begin{bmatrix} -1. & 0.5 \end{bmatrix}$

$\begin{bmatrix} 0.75 & -0.25 \end{bmatrix}$

Experiment 3: Load, Clean and Analyze Dataset using Pandas

A student dataset contains missing values. Write a Python program using Pandas to load the dataset, clean missing values, and calculate statistical information such as average marks and highest score.

Aim

To load a dataset, clean missing values, and calculate statistical information using Pandas.

Algorithm

- Import the Pandas library.
- Create or load the dataset.
- Display the original dataset.
- Replace missing values with the average value.
- Display the cleaned dataset.
- Calculate the average marks.
- Find the highest marks.
- Display the results.

Program

```
import pandas as pd

data = {
    "Name": ["Alice", "Bob", "Charlie", "David"],
    "Marks": [85, None, 90, 78]
}

df = pd.DataFrame(data)

print("Original Dataset")
print(df)

# Fill missing values
df["Marks"] = df["Marks"].fillna(df["Marks"].mean())

print("\nCleaned Dataset")
print(df)

print("\nAverage Marks:", df["Marks"].mean())
print("Highest Marks:", df["Marks"].max())
```

Input

Name	Marks
------	-------

Alice	85
-------	----

Bob	NaN
-----	-----

Charlie	90
---------	----

David	78
-------	----

Output

Cleaned Dataset

Name	Marks
------	-------

Alice	85.0
-------	------

Bob	84.33
-----	-------

Charlie	90.0
---------	------

David	78.0
-------	------

Average Marks: 84.33

Highest Marks: 90.0

Result

The dataset was successfully cleaned, and the average and highest marks were calculated.

Experiment 4: Data Visualization using Matplotlib

The sales of a shop for five months are given. Write a Python program using Matplotlib to visualize the monthly sales using a bar chart and line graph.

Aim

To visualize monthly sales data using a bar chart and a line graph with Matplotlib.

Algorithm

- Import the Matplotlib library.
- Define the months and sales values.
- Plot the bar chart.
- Display the bar chart.
- Plot the line graph.
- Display the line graph.

Program

```
import matplotlib.pyplot as plt

months = ["Jan", "Feb", "Mar", "Apr", "May"]
sales = [200, 350, 300, 450, 500]

# Bar Chart
plt.bar(months, sales)
plt.title("Monthly Sales")
plt.xlabel("Months")
plt.ylabel("Sales")
plt.show()

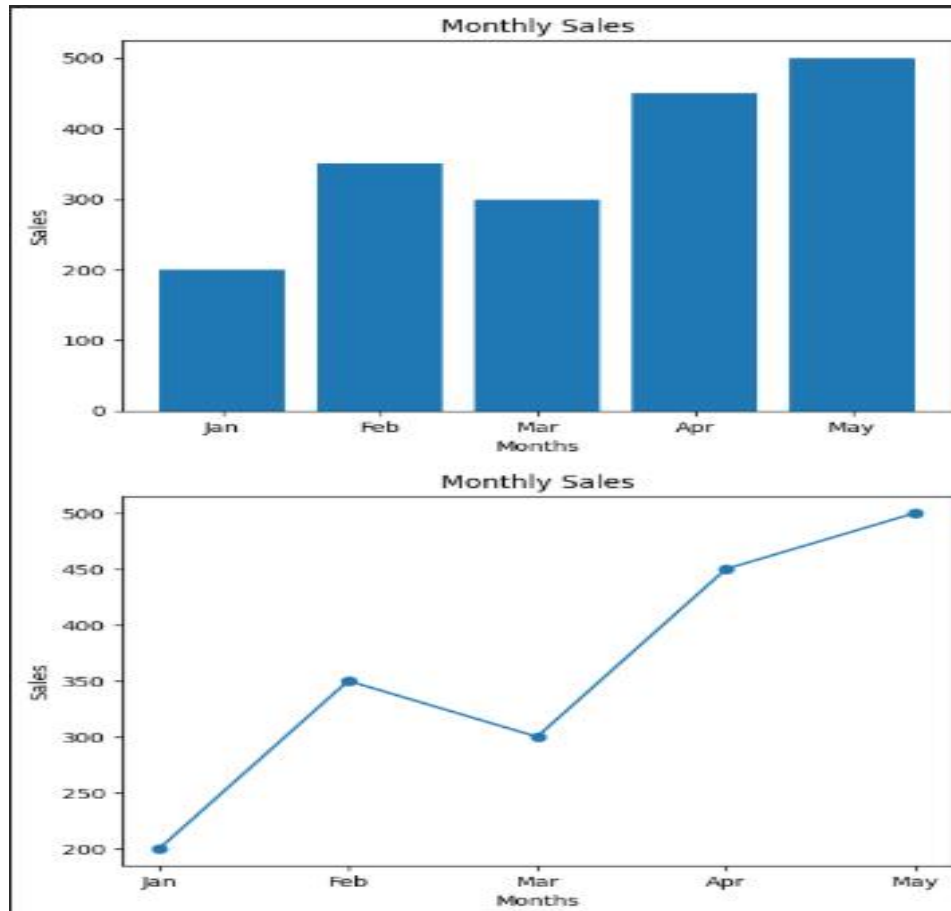
# Line Graph
plt.plot(months, sales, marker='o')
plt.title("Monthly Sales")
plt.xlabel("Months")
plt.ylabel("Sales")
plt.show()
```

Input

Months = Jan, Feb, Mar, Apr, May

Sales = 200, 350, 300, 450, 500

Output



Result

The monthly sales data was successfully visualized using both bar and line charts.

Experiment 5: Cosine Similarity between Two Text Vectors

Two documents are represented as numerical vectors. Write a Python program to calculate the cosine similarity between the two vectors using Scikit-learn and interpret the similarity score.

Aim

To calculate the cosine similarity between two text documents using Scikit-learn.

Algorithm

- Import the required libraries.
- Create two text documents.
- Convert the text into numerical vectors using CountVectorizer.
- Calculate cosine similarity using `cosine_similarity()`.
- Display the similarity matrix and similarity score.

Program

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics.pairwise import cosine_similarity

documents = [
    "Artificial Intelligence is the future",
    "AI is the future of technology"
]

vectorizer = CountVectorizer()

vectors = vectorizer.fit_transform(documents)

similarity = cosine_similarity(vectors)

print("Cosine Similarity Matrix:")
print(similarity)

print("\nSimilarity Score:", similarity[0][1])
```

Input

Document 1: Artificial Intelligence is the future

Document 2: AI is the future of technology

Output

Cosine Similarity Matrix:

```
[[1.      0.54772256]
```

```
 [0.54772256    1.]]
```

Similarity Score: 0.5477225575051662

Result

The cosine similarity between the two text documents was successfully calculated using Scikit-learn.